

Unidad 5: Acceso a estructura NoSQL

Clase 11 – Arquitecturas híbridas y persistencia polimórfica

Objetivo

Diseñar soluciones híbridas que integren múltiples fuentes de datos relacionales y NoSQL, aplicando conceptos de persistencia polimórfica y evaluando su impacto en sistemas reales.

Tema

Persistencia polimórfica.
Acceso a múltiples fuentes de datos.
Diseño de soluciones híbridas (SQL + NoSQL).
Integración de tecnologías del TP.

Arquitecturas híbridas y persistencia políglota en Ingeniería de Datos II

Introducción conceptual

Las arquitecturas híbridas de datos surgen como respuesta a una necesidad cada vez más frecuente en los sistemas actuales: integrar distintos tipos de información, con diferentes niveles de estructura, volumen, velocidad, criticidad y formas de consulta. En un sistema empresarial real, no todos los datos tienen la misma naturaleza ni cumplen la misma función. Algunos requieren integridad transaccional estricta, otros necesitan flexibilidad documental, otros deben responder en milisegundos, otros se generan en grandes volúmenes como eventos o logs, y otros adquieren valor cuando se analizan como relaciones complejas entre entidades.

La persistencia polimórfica, también llamada persistencia políglota, consiste en utilizar diferentes modelos y motores de bases de datos dentro de una misma solución, asignando a cada uno una responsabilidad específica según el tipo de dato y el uso esperado. La idea central no es acumular tecnologías, sino elegir conscientemente la herramienta más adecuada para cada necesidad del sistema.

En una plataforma integral de gestión médica, por ejemplo, conviven datos de pacientes, médicos, centros de atención, turnos, reservas, facturación, sesiones, auditoría, eventos técnicos, formularios clínicos y relaciones entre tratamientos, profesionales y derivaciones. Pretender resolver todo con una única base de datos puede generar diseños rígidos, consultas ineficientes, problemas de escalabilidad, duplicación descontrolada o pérdida de trazabilidad. Por eso, una arquitectura híbrida bien diseñada permite organizar mejor las responsabilidades: qué motor conserva la fuente de verdad, cuál administra estados temporales, cuál registra eventos masivos,

cuál permite análisis de relaciones y cuál sostiene las operaciones centrales del negocio.

El problema profesional que permite resolver este enfoque es el diseño de soluciones de datos más realistas, escalables y alineadas con las necesidades del negocio. No se trata solamente de conocer SQL Server, MongoDB, Redis, Cassandra o Neo4j de manera aislada, sino de comprender cómo pueden convivir dentro de una arquitectura coherente.

Marco conceptual principal

Persistencia polimórfica o persistencia políglota

La persistencia polimórfica es un enfoque de diseño en el cual una aplicación utiliza más de un mecanismo de almacenamiento, seleccionando cada motor según el tipo de información que debe administrar. El término “políglota” remite a la capacidad de hablar varios lenguajes; aplicado a datos, significa que una solución puede “hablar” el lenguaje relacional, documental, clave-valor, wide-column o grafo, según corresponda.

Este enfoque parte de una premisa fundamental: ningún motor de base de datos es óptimo para todos los problemas. Una base relacional es excelente para garantizar integridad, normalización, restricciones, transacciones y consultas estructuradas. Una base documental es adecuada para datos flexibles o semiestructurados. Una base clave-valor ofrece muy baja latencia para estados simples y temporales. Una base wide-column permite manejar grandes volúmenes de escritura y consultas previamente diseñadas. Una base de grafos facilita el análisis de vínculos, caminos y patrones relacionales complejos.

La persistencia polimórfica no debe confundirse con dispersión tecnológica. Usar varios motores sin una justificación clara puede aumentar la complejidad, dificultar la administración, generar duplicación de datos y volver más costoso el mantenimiento. La decisión debe surgir de una evaluación técnica y funcional: qué dato se guarda, quién lo consulta, con qué frecuencia, qué consistencia requiere, cuánto volumen genera, qué latencia tolera y qué valor aporta al proceso.

Arquitectura híbrida de datos

Una arquitectura híbrida de datos es una estructura de solución que integra diferentes tecnologías de almacenamiento, procesamiento e intercambio de información. Puede incluir bases relacionales, bases NoSQL, cachés, colas de eventos, servicios de integración, aplicaciones, interfaces de programación de aplicaciones y herramientas analíticas.

El concepto de NoSQL hace referencia a modelos de bases de datos no relacionales o no exclusivamente relacionales. No implica ausencia total de estructura, sino una forma distinta de organizar y consultar los datos. MongoDB trabaja con documentos, Redis con claves y estructuras en memoria, Cassandra con tablas orientadas a consultas distribuidas y Neo4j con nodos y relaciones.

Una arquitectura híbrida debe responder preguntas concretas:

Dónde se guarda cada dato.

Cuál es la fuente de verdad.

Qué datos se duplican y por qué.

Qué operaciones requieren consistencia inmediata.

Qué procesos pueden trabajar con consistencia eventual.

Qué motor resuelve mejor cada patrón de consulta.

Cómo se mantiene la trazabilidad.

Cómo se integran los componentes.

Qué ocurre si un motor falla o queda desactualizado.

Por lo tanto, el diseño híbrido no se mide por la cantidad de tecnologías utilizadas, sino por la claridad de sus responsabilidades.

Fuente de verdad, replicación y sincronización

Uno de los conceptos más importantes en una arquitectura híbrida es la fuente de verdad. La fuente de verdad es el repositorio considerado oficial para un dato determinado. Por ejemplo, en una plataforma médica, SQL Server puede ser la fuente de verdad para pacientes, médicos, centros médicos, obras sociales y facturación. MongoDB puede guardar documentos clínicos relacionados, Cassandra puede registrar eventos de actividad y Neo4j puede representar relaciones derivadas para análisis.

La replicación de datos consiste en copiar información entre nodos o sistemas con fines de disponibilidad, rendimiento, tolerancia a fallos o análisis. La sincronización, en cambio, es el proceso que mantiene alineadas esas copias o representaciones. Puede ser inmediata, diferida, por lotes o basada en eventos.

Esto es crítico porque en una arquitectura híbrida puede existir el mismo dato representado en diferentes lugares. Por ejemplo, el identificador de un paciente puede aparecer en SQL Server, en una reserva de MongoDB, en un evento de Cassandra y como nodo en Neo4j. Sin embargo, eso no significa que todos esos motores sean dueños del dato. Debe existir una definición clara de autoridad. Si el nombre o la cobertura médica del paciente cambia, el sistema debe saber cuál es la fuente oficial y cómo se actualizan las demás representaciones.

Consistencia y trazabilidad

La consistencia indica el grado de alineación entre los datos después de una operación. En algunos casos se necesita consistencia fuerte: una factura emitida, un pago confirmado o una reserva finalizada no deberían quedar en estados contradictorios. En otros casos puede aceptarse consistencia eventual: un tablero analítico, una

recomendación, un ranking o una vista de relaciones puede actualizarse con cierto retraso sin afectar la operación principal.

La trazabilidad es la capacidad de reconstruir lo ocurrido. Permite responder quién hizo una operación, cuándo, desde dónde, sobre qué dato, con qué resultado y qué impacto generó. En sistemas médicos, la trazabilidad es especialmente importante porque involucra datos sensibles, auditoría, responsabilidades profesionales y seguimiento de procesos.

No debe confundirse estado actual con evento histórico. El estado actual de un turno puede indicar “confirmado”, mientras que los eventos históricos pueden mostrar que primero fue solicitado, luego bloqueado temporalmente, después confirmado, modificado y finalmente atendido. El estado resume la situación actual; los eventos explican el camino recorrido.

Sistema transaccional y sistema analítico

Un sistema transaccional es aquel que soporta las operaciones centrales del negocio. En una red médica, esto incluye registrar pacientes, asignar médicos, confirmar turnos, emitir facturas, validar obras sociales o guardar operaciones administrativas. Estos procesos requieren integridad, control y consistencia.

Un sistema analítico se orienta a explorar datos, detectar patrones, generar indicadores, estudiar comportamientos y apoyar la toma de decisiones. Puede responder preguntas como: qué centros tienen mayor demanda, qué especialidades presentan más cancelaciones, qué médicos derivan más pacientes, qué horarios concentran mayor actividad o qué eventos técnicos anticipan fallas operativas.

Ambos enfoques son necesarios, pero no tienen las mismas exigencias. Un sistema transaccional prioriza confiabilidad operativa. Un sistema analítico prioriza explotación, lectura, agregación y descubrimiento. Una arquitectura híbrida debe reconocer esta diferencia y no forzar a un único motor a resolverlo todo.

Modelo orientado a consultas

El modelo orientado a consultas consiste en diseñar la estructura de datos a partir de las preguntas que el sistema necesita responder. Este criterio es especialmente importante en Cassandra y también en MongoDB cuando se busca rendimiento. A diferencia del enfoque relacional tradicional, donde se normaliza el modelo y luego se escriben consultas sobre las tablas, en algunos motores NoSQL se define primero la consulta y luego se diseña la estructura que la soporta eficientemente.

Por ejemplo, si se necesita consultar eventos de un paciente ordenados por fecha, Cassandra podría diseñar una tabla particionada por paciente y ordenada por fecha. Si se necesita consultar eventos por centro médico y día, probablemente se necesite otra tabla específica. Esto puede implicar duplicación controlada, pero orientada a resolver consultas de alto volumen de manera eficiente.

Aplicación profesional

En Ingeniería de Datos II, las arquitecturas híbridas permiten comprender cómo se diseñan soluciones modernas donde conviven operación, análisis, integración y escalabilidad. Este enfoque tiene aplicación directa en sistemas de salud, comercio electrónico, logística, banca, educación, auditoría, ciberseguridad y plataformas digitales.

En el caso de una red médica, la aplicación podría tener una capa de servicios desarrollada en Python. Python actuaría como coordinador de acceso a datos, utilizando conectores o drivers para comunicarse con cada motor. El diseño no debería limitarse a “conectar bases”, sino a definir responsabilidades.

Un posible reparto sería el siguiente:

SQL Server administra pacientes, médicos, centros, obras sociales, contratos, facturación y reglas administrativas.

MongoDB almacena reservas extendidas, formularios clínicos, notas de atención y documentos semiestructurados.

Redis gestiona sesiones, caché, bloqueos temporales de turnos y validaciones rápidas.

Cassandra registra eventos masivos, actividad de usuarios, logs técnicos y trazabilidad por fecha.

Neo4j representa relaciones entre pacientes, médicos, especialidades, centros, tratamientos y derivaciones.

La aplicación profesional del enfoque aparece cuando se diseña un flujo completo. Por ejemplo, para reservar un turno, el sistema puede consultar disponibilidad en SQL Server, bloquear temporalmente el turno en Redis, guardar detalles flexibles de la reserva en MongoDB, registrar eventos en Cassandra y luego alimentar un grafo para analizar patrones de derivación o comportamiento.

Relación con el uso políglota de bases de datos, integración, intercambio y persistencia

El uso políglota de bases de datos requiere pensar la persistencia como una arquitectura distribuida de responsabilidades. Cada motor conserva información, pero no todos cumplen el mismo rol. La integración es el conjunto de mecanismos que permite que esos motores colaboren dentro de un flujo coherente. El intercambio de datos define cómo circula la información entre componentes. La persistencia define dónde y cómo se conserva cada dato.

En una solución profesional, la integración puede realizarse mediante servicios de aplicación, procesos por lotes, eventos, colas, interfaces de programación de aplicaciones o pipelines de datos. Una interfaz de programación de aplicaciones, conocida como API por sus siglas en inglés Application Programming Interface, permite que diferentes componentes intercambien información mediante contratos definidos.

La persistencia políglota obliga a documentar decisiones. No alcanza con decir “usamos MongoDB” o “usamos Cassandra”. Es necesario justificar:

Qué dato se guarda allí.

Qué consulta resuelve.

Qué volumen espera manejar.

Qué consistencia necesita.

Qué relación tiene con otros datos.

Qué ocurre si el dato queda desactualizado.

Qué motor es fuente de verdad.

Qué riesgos introduce esa decisión.

Esta forma de análisis es especialmente relevante para el Trabajo Práctico, porque permite pasar de una descripción tecnológica a una propuesta arquitectónica defendible.

Ejemplos de integración y codificación básica

Los siguientes fragmentos son ejemplos simples y representativos. No constituyen una solución completa, pero muestran cómo una aplicación puede interactuar con distintos motores dentro de un mismo flujo.

Acceso básico desde Python a MongoDB

MongoDB puede utilizarse para guardar una reserva médica con estructura flexible. Por ejemplo, una reserva puede tener datos administrativos, observaciones y campos específicos según la especialidad.

```
from pymongo import MongoClient
from datetime import datetime
```

```
cliente = MongoClient("mongodb://localhost:27017/")
db = cliente["red_medica"]
reservas = db["reservas"]
```

```
reserva = {
    "id_reserva": "RES-10045",
    "id_paciente": 501,
    "id_medico": 32,
    "especialidad": "Cardiología",
    "centro": "Centro Médico Norte",
    "fecha_turno": "2026-06-10",
    "estado": "pendiente_confirmacion",
```

```
"formulario_previo": {
  "motivo_consulta": "control anual",
  "antecedentes": ["hipertension"],
  "observaciones": "Paciente solicita turno por la mañana"
},
"fecha_creacion": datetime.utcnow()
}
```

```
reservas.insert_one(reserva)
print("Reserva guardada en MongoDB")
```

La ventaja de este modelo es que el formulario previo puede variar según la especialidad. No todos los documentos necesitan tener exactamente la misma estructura.

Uso básico de Redis para bloqueo temporal de turno

Redis puede utilizarse para evitar que dos pacientes reserven el mismo turno al mismo tiempo. El bloqueo puede tener un tiempo de vida limitado mediante TTL, del inglés Time To Live, que indica cuánto tiempo permanece activa una clave.

```
import redis
```

```
r = redis.Redis(host="localhost", port=6379, db=0, decode_responses=True)
```

```
id_turno = "turno:bloqueado:MED32:2026-06-10:09:00"
```

```
bloqueo = r.set(id_turno, "paciente_501", nx=True, ex=300)
```

```
if bloqueo:
```

```
    print("Turno bloqueado temporalmente por 5 minutos")
```

```
else:
```

```
    print("El turno ya se encuentra bloqueado por otro usuario")
```

La opción `nx=True` indica que la clave solo se crea si no existe. La opción `ex=300` define una expiración de 300 segundos. Redis no debería ser la fuente final de la reserva confirmada, pero sí es muy útil para controlar concurrencia de baja latencia.

Tabla CQL para Cassandra

Cassandra Query Language, conocido como CQL, es el lenguaje utilizado para trabajar con Cassandra. En Cassandra, la tabla se diseña según la consulta esperada. Si se necesita consultar eventos de un paciente por fecha, una posible tabla sería:

```
CREATE TABLE eventos_por_paciente (
  id_paciente int,
  fecha_evento date,
  timestamp_evento timestamp,
```

```

    tipo_evento text,
    id_referencia text,
    detalle text,
    PRIMARY KEY ((id_paciente), fecha_evento, timestamp_evento)
) WITH CLUSTERING ORDER BY (fecha_evento DESC, timestamp_evento DESC);

```

Ejemplo de inserción:

```

INSERT INTO eventos_por_paciente (
    id_paciente,
    fecha_evento,
    timestamp_evento,
    tipo_evento,
    id_referencia,
    detalle
)
VALUES (
    501,
    '2026-06-10',
    toTimestamp(now()),
    'RESERVA_CONFIRMADA',
    'RES-10045',
    'El paciente confirmó un turno con cardiología'
);

```

Consulta representativa:

```

SELECT fecha_evento, timestamp_evento, tipo_evento, detalle
FROM eventos_por_paciente
WHERE id_paciente = 501;

```

Esta tabla no intenta representar todo el modelo médico. Está diseñada para responder eficientemente una consulta específica: recuperar la actividad de un paciente.

Consulta Cypher en Neo4j

Neo4j utiliza Cypher como lenguaje de consulta para grafos. Un grafo puede representar pacientes, médicos, especialidades y centros mediante nodos y relaciones.

Ejemplo conceptual:

```

MATCH (p:Paciente {id_paciente: 501})-[:FUE_ATENDIDO_POR]->(m:Medico)-
[:ESPECIALISTA_EN]->(e:Especialidad)
RETURN p.nombre AS paciente, m.nombre AS medico, e.nombre AS especialidad;

```

Otra consulta posible para detectar derivaciones frecuentes entre médicos:

```

MATCH (m1:Medico)-[d:DERIVA_A]->(m2:Medico)
WHERE d.cantidad_derivaciones > 10

```

```
RETURN m1.nombre AS medico_origen,
       m2.nombre AS medico_destino,
       d.cantidad_derivaciones AS derivaciones
ORDER BY derivaciones DESC;
```

Este tipo de consulta es valiosa cuando el objetivo es analizar relaciones, caminos o patrones que no son simples filas aisladas.

Integración básica de flujo en Python

Un pseudoflujo simplificado podría representar la coordinación entre Redis, MongoDB y Cassandra:

```
def reservar_turno(id_paciente, id_medico, fecha, hora):
    clave_bloqueo = f"turno:bloqueado:{id_medico}:{fecha}:{hora}"

    bloqueo = redis_client.set(clave_bloqueo, id_paciente, nx=True, ex=300)

    if not bloqueo:
        return "Turno no disponible temporalmente"

    reserva = {
        "id_paciente": id_paciente,
        "id_medico": id_medico,
        "fecha": fecha,
        "hora": hora,
        "estado": "pendiente_confirmacion"
    }

    mongo_db.reservas.insert_one(reserva)

    cassandra_session.execute("""
        INSERT INTO eventos_por_paciente
        (id_paciente, fecha_evento, timestamp_evento, tipo_evento, id_referencia,
detalle)
        VALUES (%s, %s, toTimestamp(now()), %s, %s, %s)
        """, (
        id_paciente,
        fecha,
        "RESERVA_INICIADA",
        f"{id_medico}-{fecha}-{hora}",
        "Inicio de proceso de reserva"
        ))

    return "Reserva iniciada correctamente"
```

Este ejemplo muestra una idea central: la aplicación coordina, pero cada motor cumple una función diferente.

Buenas prácticas

Una buena arquitectura híbrida debe comenzar por el análisis de datos y consultas, no por la selección de herramientas. Primero se identifican los procesos, entidades, eventos, documentos, relaciones y necesidades analíticas. Luego se decide qué motor resuelve mejor cada caso.

Conviene definir explícitamente la fuente de verdad de cada dato. Si SQL Server es la fuente oficial de pacientes y médicos, MongoDB, Cassandra o Neo4j pueden conservar referencias, copias parciales o representaciones analíticas, pero no deberían convertirse en autoridades contradictorias.

También es recomendable documentar los flujos de sincronización. Cada dato que se replica o deriva debe tener una regla clara: cuándo se actualiza, quién lo actualiza, qué ocurre si falla la actualización y cómo se detectan inconsistencias.

Otra buena práctica es separar estado actual, historial y evento. El estado actual permite operar; el historial permite auditar; el evento permite reconstruir trazabilidad. Mezclar estos conceptos puede generar diseños confusos.

En Redis, deben usarse claves claras, tiempos de expiración y estructuras simples. Redis es muy útil para sesiones, bloqueos, caché y contadores rápidos, pero no debería contener información crítica sin persistencia complementaria.

En Cassandra, las tablas deben diseñarse según las consultas. No se debe esperar trabajar con joins ni consultas ad hoc como en SQL. Cada tabla debe tener una razón de negocio y una consulta principal asociada.

En MongoDB, los documentos deben diseñarse según el acceso esperado. No conviene fragmentar excesivamente la información si luego siempre se consulta junta. Tampoco conviene crear documentos gigantes o sin control de estructura.

En Neo4j, el diseño debe centrarse en relaciones relevantes. No toda relación justifica un grafo. El grafo aporta valor cuando interesa explorar caminos, vínculos indirectos, comunidades, patrones anómalos o relaciones de muchos saltos.

Finalmente, toda arquitectura híbrida debe contemplar seguridad, monitoreo, auditoría y gobierno de datos. En sistemas médicos, esto es especialmente importante por la sensibilidad de la información personal y clínica.

Errores frecuentes y riesgos

Un error habitual es elegir tecnologías por moda. Usar MongoDB, Redis, Cassandra o Neo4j no mejora automáticamente una solución. Si no existe una necesidad concreta, el resultado puede ser una arquitectura más compleja, más difícil de mantener y más costosa.

Otro error frecuente es no definir la fuente de verdad. Si un dato aparece en varios motores y ninguno es reconocido como oficial, pueden aparecer contradicciones difíciles de resolver. Por ejemplo, un paciente podría figurar con un plan de obra social en SQL Server y con otro en MongoDB.

También es riesgoso duplicar datos sin estrategia de sincronización. La duplicación puede ser correcta cuando mejora rendimiento o permite análisis, pero debe estar controlada. De lo contrario, los datos se vuelven inconsistentes.

Un error técnico común es intentar usar Cassandra como si fuera una base relacional. Cassandra no está pensada para joins complejos, normalización tradicional ni consultas libres. Su potencia aparece cuando las consultas están previstas y el modelo se diseña para ellas.

En MongoDB, un error frecuente es guardar cualquier cosa sin criterio documental. La flexibilidad no debe confundirse con desorden. Los documentos deben tener una estructura razonable, validaciones mínimas y coherencia funcional.

En Redis, el riesgo principal es usarlo como almacenamiento definitivo de datos críticos. Si un bloqueo temporal se pierde, puede regenerarse; si una reserva confirmada solo existe en Redis, el sistema queda expuesto a inconsistencias graves.

En Neo4j, el error frecuente es representar en grafo relaciones que no necesitan análisis relacional profundo. Esto agrega complejidad sin aportar valor. El grafo debe usarse cuando las relaciones son parte central de la pregunta de negocio.

Otro riesgo importante es subestimar la operación. Una arquitectura híbrida requiere monitoreo, copias de seguridad, políticas de seguridad, control de accesos, manejo de errores, observabilidad y documentación. Cuantos más componentes existen, mayor es la necesidad de gobierno técnico.

Ejemplos aplicados

Ejemplo aplicado: red médica con reserva de turnos y trazabilidad

Una red de clínicas con varios centros médicos necesita gestionar turnos en línea. El sistema debe permitir que un paciente reserve un turno con un especialista, complete un formulario previo, reciba confirmación y luego sea atendido. Al mismo tiempo, la organización necesita auditar el proceso y analizar la actividad por paciente, médico y centro.

En este caso, SQL Server puede administrar los datos maestros: pacientes, médicos, especialidades, centros, obras sociales, horarios y facturación. Redis puede bloquear temporalmente un turno mientras el paciente confirma la operación. MongoDB puede guardar el documento de reserva con el formulario previo, que puede variar según la especialidad. Cassandra puede registrar los eventos de actividad: inicio de reserva, bloqueo, confirmación, cancelación, modificación o atención. Neo4j puede representar vínculos entre pacientes, médicos, especialidades y centros para analizar patrones de derivación o concentración de atención.

El valor de la arquitectura híbrida aparece porque cada motor resuelve un problema distinto. SQL Server conserva integridad; Redis evita conflictos de concurrencia; MongoDB aporta flexibilidad; Cassandra permite trazabilidad masiva; Neo4j permite descubrir relaciones. Si se intentara resolver todo con una única base, el diseño perdería eficiencia o flexibilidad en alguna dimensión.

Ejemplo aplicado: detección de circuitos anómalos de derivación

Una organización médica quiere detectar si existen circuitos inusuales de derivación entre médicos, especialidades, centros y determinados tratamientos. El objetivo no es solamente saber cuántos turnos hubo, sino comprender cómo se relacionan los actores del sistema.

SQL Server puede indicar qué médicos existen, qué especialidades tienen y en qué centros atienden. Cassandra puede registrar eventos de atención y derivación a gran escala. MongoDB puede conservar documentos clínicos o administrativos asociados a cada caso. Neo4j puede transformar esa información en un grafo donde los nodos sean médicos, pacientes, centros, especialidades y tratamientos, y las relaciones representen atención, derivación, indicación o seguimiento.

Una consulta en grafo podría detectar médicos que derivan con frecuencia a un mismo profesional o centro, pacientes que recorren circuitos repetitivos o especialidades que concentran derivaciones atípicas. Este análisis sería más complejo si se resolviera únicamente con tablas relacionales o documentos aislados, porque el valor está en los caminos y relaciones, no solo en los registros individuales.

Aplicación al Trabajo Práctico

En el Trabajo Práctico, este enfoque permite construir propuestas más sólidas y profesionales. No alcanza con mencionar que se usará una base relacional, una documental o una de grafos. Cada decisión debe estar justificada por el problema que resuelve.

Una propuesta bien desarrollada debería explicar qué datos son maestros, qué datos son operativos, qué datos son temporales, qué datos son eventos, qué datos tienen valor relacional y qué datos alimentan análisis. También debería indicar qué motor se utiliza para cada grupo de datos y qué consultas principales debe soportar.

Por ejemplo, si un grupo propone una plataforma de salud, comercio electrónico, logística o auditoría, debería poder justificar:

Por qué los datos administrativos o transaccionales requieren un modelo relacional.

Por qué ciertos documentos necesitan flexibilidad.

Por qué algunos estados deben manejarse en memoria con expiración.

Por qué los eventos masivos requieren una estructura escalable.

Por qué ciertos análisis de relaciones justifican un grafo.

Qué motor es fuente de verdad.

Qué información se replica o deriva.

Qué consistencia necesita cada operación.

Qué riesgos técnicos aparecen y cómo se controlan.

Este nivel de análisis eleva el trabajo desde una simple selección de herramientas hacia un diseño de arquitectura de datos.

Síntesis conceptual final

Las arquitecturas híbridas y la persistencia polimórfica representan una forma profesional de diseñar sistemas de datos en contextos reales. Su valor no está en usar muchas tecnologías, sino en asignar responsabilidades de persistencia según la naturaleza del dato, el patrón de consulta, el volumen, la velocidad, la criticidad y la necesidad de análisis.

SQL Server puede sostener el núcleo estructurado y transaccional de la organización. MongoDB puede administrar documentos flexibles y semiestructurados. Redis puede resolver estados temporales y operaciones de baja latencia. Cassandra puede registrar eventos masivos y trazabilidad distribuida. Neo4j puede analizar relaciones complejas mediante grafos y consultas Cypher.

La aplicación actúa como coordinadora del acceso, pero el diseño arquitectónico define qué dato vive en cada motor, cuál es la fuente de verdad, qué información puede duplicarse, qué procesos requieren sincronización, qué consistencia se necesita y cómo se mantiene la trazabilidad.

Este enfoque aporta una mirada más madura para la dirección técnica de proyectos de datos. Permite diseñar soluciones escalables, justificadas y alineadas con necesidades reales, evitando tanto el reduccionismo de querer resolver todo con una única base como el exceso de incorporar tecnologías sin criterio. Una arquitectura híbrida bien diseñada no acumula motores: organiza responsabilidades, reduce tensiones técnicas y mejora la capacidad del sistema para operar, auditar, analizar y evolucionar.